MVC Architecture - Complete Notes + Interview Q&A

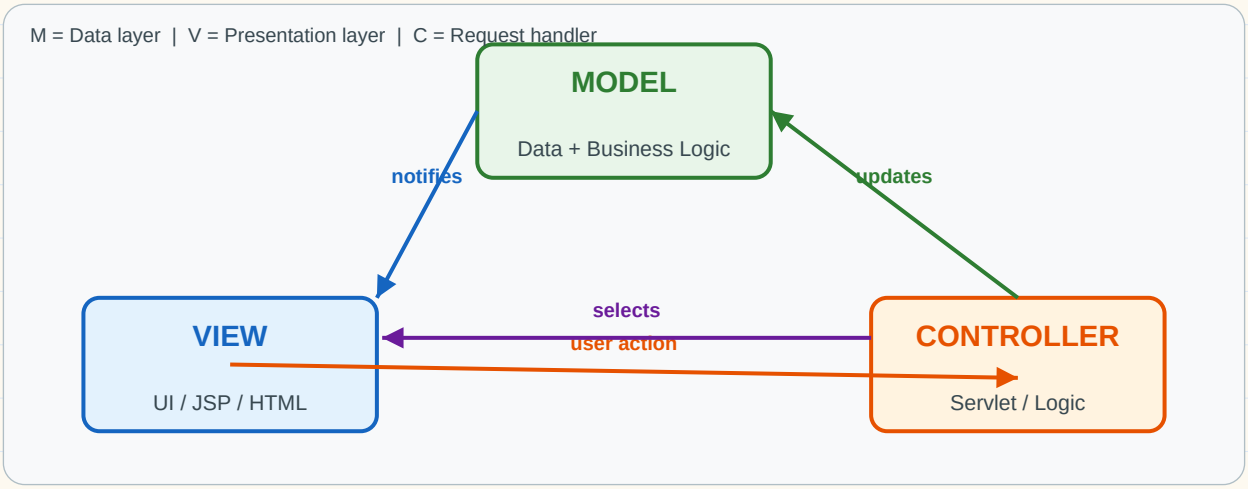
Covers: MVC Pattern - Components - Request Flow - MVC with Servlets & JSP - Spring MVC - Design Patterns - Top Company Interview Questions

Part 1 - What is MVC? Core Concept

MVC = Model - View - Controller

- A software DESIGN PATTERN that separates an application into 3 interconnected components
- Goal: Separation of Concerns - each layer has ONE job, nothing more
- Originated from Smalltalk-80 (1970s), now used in almost every web framework
- Key benefit: easier to maintain, test, and scale independently
- Used in: Spring MVC, Struts, ASP.NET MVC, Django, Ruby on Rails, Angular

The MVC Diagram:



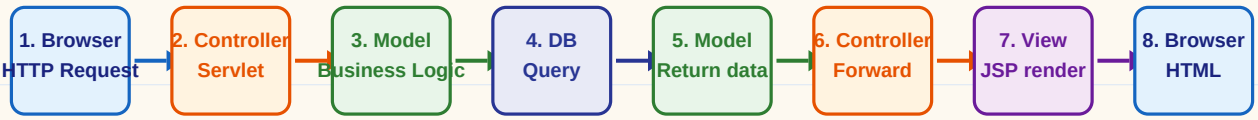
The 3 Components Explained:

Component	Responsibility	In Java Web	Examples
MODEL (M)	- Holds application data - Business logic & rules - Database interaction - Data validation	POJO / JavaBean DAO classes Hibernate entities Service classes	User.java ProductDAO.java OrderService.java
VIEW (V)	- Displays data to user - UI presentation only - No business logic here - Reads from Model only	JSP pages HTML templates Thymeleaf FreeMarker	login.jsp home.jsp product.html
CONTROLLER (C)	Handles user requests - Calls Model for data - Selects which View - Glue between M and V	Servlets Spring @Controller Action classes	LoginServlet.java HomeController.java

Key Principle: Model knows nothing about View. View knows nothing about Controller. Controller knows both.

Part 2 - MVC Request Flow (Step by Step)

Complete Request-Response Flow:



Detailed Step Walkthrough:

Step	What Happens
Step 1	User submits a form or clicks a link - Browser sends HTTP request to server
Step 2	Controller (Servlet) receives request - reads parameters via request.getParameter()
Step 3	Controller calls the Model (Service/DAO layer) to perform business logic / DB query
Step 4	Model fetches data from Database (JDBC/Hibernate), processes it, returns result object
Step 5	Controller receives data - stores in request scope: request.setAttribute('key', data)
Step 6	Controller selects View - forwards to JSP using RequestDispatcher.forward(req, res)
Step 7	View (JSP) reads data via request.getAttribute('key') - renders HTML dynamically
Step 8	Final HTML response sent back to browser - user sees the result page

Part 3 - Implementing MVC with Servlet + JSP (Full Code)

1. Model - User.java (POJO)

```
// User.java - Model
```

```
// MODEL: Plain Java class - holds data, no HTTP knowledge
public class User {
    private int id;
    private String name;
    private String email;

    // Constructor
    public User(int id, String name, String email) {
        this.id = id; this.name = name; this.email = email;
    }
    // Getters & Setters (standard JavaBean pattern)
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    // ... other getters/setters
}
```

2. Model - UserDAO.java (Data Access Object)

```
// UserDAO.java - Data Access Layer

// DAO: Handles all DB operations - JDBC code lives here only
public class UserDAO {
    public User getUserById(int id) {
        User user = null;
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM users WHERE id = ?");
            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                user = new User(rs.getInt("id"),
                                rs.getString("name"),
                                rs.getString("email"));
            }
        } catch (SQLException e) { e.printStackTrace(); }
        return user;
    }
    public List<User> getAllUsers() { /* similar pattern */ }
}
```

3. Controller - UserController.java (Servlet)

```
// UserController.java - Controller

// CONTROLLER: Receives request, calls model, selects view
@WebServlet("/user")
public class UserController extends HttpServlet {

    private UserDAO userDAO = new UserDAO(); // Model reference

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        // 1. Read input from request
        String idStr = request.getParameter("id");
        int id = Integer.parseInt(idStr);

        // 2. Call Model (business logic)
        User user = userDAO.getUserById(id); // Model does DB work

        // 3. Store result in request scope
        request.setAttribute("user", user); // Pass to View

        // 4. Select View and forward
        RequestDispatcher rd =
            request.getRequestDispatcher("/userProfile.jsp");
        rd.forward(request, response); // URL stays same
    }
}
```

4. View - userProfile.jsp (JSP)

```
// userProfile.jsp - View
<%-- VIEW: Only displays data - no business logic here --%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<body>
    <%
        // Retrieve model data set by Controller
        User user = (User) request.getAttribute("user");
    %>

    <h1>User Profile</h1>
    <p>Name:  <%= user.getName()  %></p>  <%-- Expression tag --%>
    <p>Email: <%= user.getEmail() %></p>

    <%-- Using JSTL (cleaner approach) --%>
    <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
    <p>Name:  <c:out value="${user.name}"  /></p>
    <p>Email: <c:out value="${user.email}" /></p>
</body></html>
```

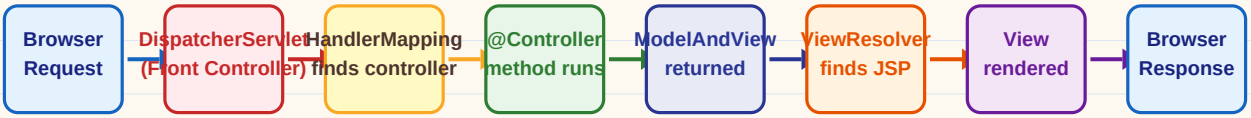
Pattern: Controller never generates HTML. View never connects to DB. Model never knows about HTTP.

Part 4 - Front Controller Pattern (Used in Spring MVC)

Front Controller Pattern

- A single entry-point Servlet/Controller handles ALL requests first
- It then delegates to appropriate sub-controllers / handlers
- Advantages: centralized security, logging, session checking, URL routing
- Used in: Spring MVC (DispatcherServlet), Struts (ActionServlet)
- The DispatcherServlet IS the Front Controller in Spring MVC

Front Controller Flow (Spring MVC):



Spring MVC Code Example:

```
// Spring MVC Controller
```

```
// Spring MVC Controller - no extends HttpServlet needed!
@Controller  // Marks this as a Spring MVC Controller
@RequestMapping("/users")  // Base URL mapping
public class UserController {

    @Autowired
    private UserService userService; // Injected by Spring (IoC)

    @GetMapping("/{id}")  // Handles GET /users/{id}
    public String getUser(@PathVariable int id, Model model) {
        User user = userService.findById(id); // Call service layer
        model.addAttribute("user", user);    // Add to model
        return "userProfile";  // Returns VIEW name (no .jsp needed)
    }

    @PostMapping("/save")  // Handles POST /users/save
    public String saveUser(@ModelAttribute User user) {
        userService.save(user);
        return "redirect:/users/list"; // PRG pattern
    }
}
```

web.xml - DispatcherServlet Configuration:

```
// web.xml - Front Controller Setup
<!-- web.xml: Register DispatcherServlet as the Front Controller -->
<web-app>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>
      org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>  <!-- Load at startup -->
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/</url-pattern>  <!-- Catch ALL requests -->
  </servlet-mapping>
</web-app>
```

Plain MVC (Servlet+JSP)	Spring MVC
- Multiple Servlets, each mapped	- Single DispatcherServlet (front ctrl)
- Manual request.getParameter()	- @RequestParam auto binding
- Manual forward/redirect code	- return 'viewName' - simple
- No dependency injection	- @Autowired DI - Spring IoC
- More boilerplate code	- Much less boilerplate
- No ViewResolver - hardcode paths	- ViewResolver resolves view names
- Good for learning concepts	- Industry standard for production

Part 5 - MVC Patterns, Layers & Best Practices

Layered Architecture (MVC + Service + DAO):

Layer	Component	Responsibility	Talks To
Presentation	JSP / HTML (View)	Display data, collect user input	Controller only
Controller	Servlet / @Controller	Route requests, validate input, call service layer	View + Service
Service	Service class (Business Logic)	Business rules, transactions, orchestrate operations	Controller + DAO
DAO / Repository	DAO class (Data Access)	CRUD operations, SQL queries, DB interaction only	Service + DB
Model / Entity	POJO / Entity JavaBean	Data container - passed between all layers	All layers
Database	MySQL / Oracle / H2	Persist and retrieve data	DAO only

Service Layer Code Example:

```
// UserService.java - Business Logic Layer
```

```
// SERVICE LAYER: Business logic lives here, not in Controller!
public class UserService {
    private UserDAO userDAO = new UserDAO();

    public User login(String email, String password) {
        User user = userDAO.findByEmail(email); // DB call
        if (user == null) throw new RuntimeException("User not found");
        if (!user.getPassword().equals(password)) {  // Business rule
            throw new RuntimeException("Invalid password");
        }
        return user; // Return to Controller
    }
    public void registerUser(User user) {
        // Business validation
        if (userDAO.emailExists(user.getEmail()))
            throw new RuntimeException("Email already registered");
        userDAO.save(user); // Delegate DB work to DAO
    }
}
```

Best Practices Checklist:

MVC Best Practices:

- Controller should be THIN - only routing, input reading, view selection
- Business logic belongs in Service layer, NOT in Controller or JSP
- DB code (SQL, JDBC) belongs ONLY in DAO layer
- JSP (View) should have ZERO business logic - use JSTL instead of scriptlets
- Use POST-Redirect-GET (PRG) pattern after form submissions to prevent re-submission
- One Controller per resource/feature (UserController, ProductController, etc.)
- Model objects (POJOs) should be serializable for session storage
- Use request scope for single-request data, session scope for user data

Common MVC Anti-Patterns (avoid these!):

- Putting SQL queries directly in JSP (fat view anti-pattern)
- Putting all logic in one God Servlet (spaghetti code)
- Directly calling DAO from Controller (skipping Service layer)
- Using out.println() for HTML in Servlet (fat controller anti-pattern)
- Storing sensitive data in cookies instead of server-side session
- Using GET requests for state-changing operations (use POST instead)

Part 6 - web.xml, Filters, Listeners & JSTL

web.xml - Deployment Descriptor (Traditional way):

```
// web.xml - Complete Example
<!-- web.xml: Old way to configure servlets (before @WebServlet) -->
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">

  <!-- Define the Servlet -->
  <servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.app.LoginServlet</servlet-class>
    <init-param>
      <param-name>dbUrl</param-name>
      <param-value>jdbc:mysql://localhost/mydb</param-value>
    </init-param>
  </servlet>

  <!-- Map Servlet to URL -->
  <servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/login</url-pattern>
  </servlet-mapping>

  <!-- Welcome file list -->
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>

  <!-- Custom error pages -->
  <error-page>
    <error-code>404</error-code>
    <location>/error404.jsp</location>
  </error-page>
</web-app>
```

Servlet Filters - Intercept Requests:

```
// AuthFilter.java - Authentication Filter

// Filter: Runs BEFORE and AFTER every request matching the pattern
@WebFilter("/*")  // Apply to ALL requests
public class AuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                         FilterChain chain) throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpSession session = req.getSession(false);

        String path = req.getRequestURI();
        boolean isLoginPage = path.endsWith("login.jsp");

        if (session != null && session.getAttribute("user") != null) {
            chain.doFilter(request, response); // User logged in - continue
        } else if (isLoginPage) {
            chain.doFilter(request, response); // Login page - allow
        } else {
            ((HttpServletResponse)response).sendRedirect("/login.jsp");
        }
    }
}
```

JSTL - JavaServer Pages Standard Tag Library:

```
// JSTL + EL - Clean View Layer

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt"  prefix="fmt" %>

<!-- c:if - conditional rendering (replaces <% if %> scriptlet) -->
<c:if test="${user != null}">
    <p>Welcome, ${user.name}!</p>
</c:if>

<!-- c:forEach - loop (replaces <% for %> scriptlet) -->
<c:forEach var="item" items="${productList}">
    <tr><td>${item.name}</td><td>${item.price}</td></tr>
</c:forEach>

<!-- c:choose - if/else/switch equivalent -->
<c:choose>
    <c:when test="${score >= 90}"><span>Grade A</span></c:when>
    <c:when test="${score >= 70}"><span>Grade B</span></c:when>
    <c:otherwise><span>Grade C</span></c:otherwise>
</c:choose>

<!-- c:out - safe output (escapes HTML entities) -->
<c:out value="${user.name}" default="Guest" />

<!-- EL (Expression Language) - direct access to scoped attributes -->
<p>${sessionScope.user.email}</p>     <!-- session scope -->
<p>${requestScope.product.price}</p>  <!-- request scope -->
<p>${param.username}</p>              <!-- request parameter -->
```

Part 7 - Top Company Interview Questions on MVC

Questions asked at: Amazon, Google, Microsoft, TCS, Infosys, Wipro, Accenture, Cognizant, HCL, Capgemini, IBM

Q1 What is MVC? Why do we use it?

TCS Infosys Wipro Accenture

Answer:

MVC = Model-View-Controller, a design pattern separating app into 3 components.
Model: data/business logic. View: UI. Controller: handles requests, glues M and V.
Benefits: Separation of concerns, easier testing, maintainability, parallel development.
Widely used in Spring MVC, Struts, ASP.NET MVC, Django, Rails.

Q2 What is the role of each component in MVC?

Amazon Google TCS Cognizant

Answer:

MODEL: Holds data (POJO/Entity), business logic, DB interaction via DAO.
VIEW: Displays data (JSP/HTML), no logic - only presentation.
CONTROLLER: Receives HTTP request, calls Model, selects View (Servlet/@Controller).
Key rule: Model knows nothing about View. Controller is the mediator.

Q3 How does a request flow in MVC with Servlet and JSP?

Infosys HCL Capgemini IBM

Answer:

1. Browser sends HTTP request 2. Controller Servlet receives it
3. Controller calls DAO/Service (Model layer) 4. Model fetches DB data
5. Controller calls request.setAttribute('key', data)
6. Controller does RequestDispatcher.forward() to JSP (View)
7. JSP reads attribute and renders HTML 8. HTML sent to browser

Q4 What is the difference between MVC1 and MVC2?

TCS Wipro Accenture

Answer:

MVC1: JSP handles both View AND Controller logic (JSP calls beans directly).
- Outdated, leads to fat JSP pages, hard to maintain
MVC2: Strict separation - Servlet is Controller, JSP is only View.
- Clean architecture, Servlet routes requests, JSP only presents data.
All modern frameworks use MVC2 pattern.

Q5 What is Front Controller Pattern? How does Spring use it?

Amazon Google Microsoft TCS

Answer:

A single Servlet/Controller receives ALL requests first (central entry point).
It then delegates to specific handlers/controllers based on URL.
Spring MVC uses DispatcherServlet as Front Controller - mapped to '/' in web.xml.
Benefits: Centralized security, logging, session checks, URL routing.

Q6 What is DispatcherServlet in Spring MVC?

Infosys Cognizant HCL

Answer:

DispatcherServlet is the Front Controller of Spring MVC framework.
It intercepts ALL incoming requests (url-pattern: /).
Internally uses HandlerMapping to find the right @Controller method,
then uses ViewResolver to find the JSP/template and renders the response.

Q7 What is the difference between @Controller and @RestController?

Amazon Google Microsoft Wipro

Answer:

@Controller: Returns a VIEW NAME (String) - Spring resolves it to a JSP/template.
Methods must use @ResponseBody to return JSON/XML data.
@RestController = @Controller + @ResponseBody on every method.
Every method returns data (JSON/XML) directly - no view resolution.
Use @Controller for MVC web apps, @RestController for REST APIs.

Q8 What is ModelAndView in Spring MVC?

TCS Infosys Accenture IBM

Answer:

ModelAndView holds both: the VIEW name + the MODEL data in one object.
Example: ModelAndView mav = new ModelAndView('userList');
mav.addObject('users', userList); return mav;
Alternatively use Model parameter: model.addAttribute('key', value); return 'view';
Both approaches are equivalent - Model+String return is more modern.

Q9 What is the difference between @RequestMapping, @GetMapping, @PostMapping?

Amazon Google Microsoft Cognizant

Answer:

@RequestMapping can handle ALL HTTP methods (GET, POST, PUT, DELETE, etc.).
@RequestMapping(value='/url', method=RequestMethod.GET)
@GetMapping is a shortcut for @RequestMapping(method=GET) - cleaner syntax.
@PostMapping, @PutMapping, @DeleteMapping are similar shortcuts.
Best practice: use specific mappings (@GetMapping, @PostMapping) for clarity.

Q10 What is @ModelAttribute in Spring MVC?

TCS HCL Capgemini

Answer:

@ModelAttribute auto-binds HTML form data to a Java object (form backing bean).
Example: public String save(@ModelAttribute User user) { ... }
Spring reads all form fields and sets them on the User object automatically.
Eliminates need for manual request.getParameter() calls for every field.
Can also be used on a method to add common data to all model views.

Q11 What is the Service layer? Why not call DAO directly from Controller?

Amazon | Infosys | IBM | Accenture

Answer:

Service layer contains business logic - it sits between Controller and DAO.
Controller should only route requests. DAO should only handle DB operations.
Service layer: validates data, applies business rules, manages transactions.
Benefit: if business rules change, only Service is modified - Controller/DAO untouched.
Also enables easier unit testing - Service can be tested without HTTP context.

Q12 What is the DAO pattern? Why use it?

TCS | Wipro | Cognizant | HCL

Answer:

DAO = Data Access Object - abstracts all database operations into one class.
Controller/Service never writes SQL - they call DAO methods like getUserById(id).
Benefits: Easy to switch DB (MySQL to Oracle) - only DAO changes.
Centralised error handling, reusable queries, easier testing with mocks.
Modern alternative: Repository pattern used in Spring Data JPA.

Q13 What is POST-Redirect-GET (PRG) pattern? Why is it important?

Amazon | Google | Microsoft | TCS

Answer:

Problem: After form POST, if user refreshes page, browser re-submits the form.
This causes duplicate data insertion (duplicate orders, payments, etc.)!
Solution: After processing POST, do sendRedirect() to a GET page instead of forward.
POST - Servlet processes - response.sendRedirect('/success') - GET /success page.
PRG prevents double-submit, makes URLs bookmarkable, shows correct URL to user.

Q14 How do you handle exceptions in MVC applications?

Infosys | HCL | Capgemini | IBM

Answer:

1. Try-catch in Controller (basic, messy for every method)
2. web.xml error-page tag: map error codes (404, 500) to error JSP pages
3. In Spring MVC: @ExceptionHandler method inside @Controller class
4. Global: @ControllerAdvice + @ExceptionHandler - handles ALL controllers
5. Spring's HandlerExceptionResolver for programmatic handling

Best practice: Global @ControllerAdvice with specific exception types.

Q15 What is the difference between include directive and include action in JSP?

TCS | Wipro | Accenture

Answer:

<%@ include file='header.jsp' %> - STATIC include at TRANSLATION time.
Content of header.jsp is merged into this page's source before compiling.
Result: single compiled Servlet class. Cannot pass parameters.
<jsp:include page='header.jsp' /> - DYNAMIC include at REQUEST time.
header.jsp is processed separately on each request. Can pass parameters.
Use directive for static content (header/footer), action for dynamic content.

Q16 What is JSTL? Why prefer it over JSP scriptlets?

Amazon Google Cognizant HCL

Answer:

JSTL = JavaServer Pages Standard Tag Library - pre-built tags for common tasks.
Provides: c:forEach, c:if, c:choose, c:out, fmt:formatDate, etc.
Why avoid scriptlets: mixing Java code in HTML is messy, hard to test,
violates MVC (logic in View layer), hard for UI designers to work with.
JSTL + EL (Expression Language) = clean, readable, testable View layer.
EL: ${user.name} automatically calls user.getName() via reflection.

Q17 What are the different scopes in JSP/Servlet?

TCS Infosys IBM Capgemini

Answer:

1. Page scope: pageContext.setAttribute() - current JSP page only
2. Request scope: request.setAttribute() - single request (forward OK)
3. Session scope: session.setAttribute() - one user, multiple requests
4. Application scope: application.setAttribute()- all users, app lifetime

Rule: Use the NARROWEST scope needed. Avoid application scope for user data.
EL access: ${pageScope.x} ${requestScope.x} ${sessionScope.x} ${applicationScope.x}

Q18 How do you implement login authentication in MVC pattern?

Amazon Microsoft Wipro Accenture

Answer:

1. User submits login form (POST /login) to Controller
2. Controller calls UserService.login(email, password)
3. Service calls UserDAO.findByEmail(email) to get user from DB
4. Service validates password (BCrypt.verify() in production)
5. If valid: Controller calls session.setAttribute('user', userObj)
6. Redirect to home page (PRG pattern)
7. AuthFilter on every request checks session.getAttribute('user') != null

Q19 What is the difference between Struts and Spring MVC?

TCS Infosys IBM

Answer:

Struts: Action-based framework, uses ActionServlet as Front Controller,
config in struts-config.xml, uses ActionForm beans, now mostly legacy.
Spring MVC: Uses DispatcherServlet, annotation-driven (@Controller, @GetMapping),
integrates with entire Spring ecosystem (DI, Security, Data, Boot).
Spring MVC is strongly preferred today - Struts is largely deprecated.
Spring Boot further simplifies Spring MVC with auto-configuration.

Q20

How does Spring Boot simplify Spring MVC?

Amazon Google Microsoft Cognizant

Answer:

Spring Boot provides auto-configuration - no need for web.xml or XML config.
@SpringBootApplication auto-configures DispatcherServlet, ViewResolver, etc.
Embedded Tomcat server - no external server deployment needed.
spring-boot-starter-web adds all needed dependencies automatically.
application.properties for simple config (server.port, spring.mvc.view.prefix).
Result: Spring MVC app runs with a single main() method!

Final - MVC Quick Cheat Sheet & Memory Aids

Concept	Key Point	Remember
`MVC Pattern`	Model=Data, View=UI, Controller=Routing	M-data, V-display, C-route
`MVC1 vs MVC2`	MVC1=JSP as Controller (old), MVC2=Servlet as Controller	MVC2 = modern standard
`Front Controller`	Single entry Servlet - DispatcherServlet in Spring	One door, many rooms
`@Controller`	Returns view name (String) for JSP rendering	Returns VIEW name
`@RestController`	@Controller + @ResponseBody - returns JSON/XML	Returns DATA, not view
`@GetMapping`	Handles HTTP GET - shortcut for @RequestMapping(GET)	GET = read data
`@PostMapping`	Handles HTTP POST - shortcut for @RequestMapping(POST)	POST = submit data
`@ModelAttribute`	Auto-binds form fields to Java object	Form to Object magic
`ModelAndView`	Holds view name + model data in one return object	View + data bundle
`Service Layer`	Business logic - between Controller and DAO	Rules live here
`DAO Pattern`	All DB operations in one class - no SQL in Controller/JSP	DB = DAO only
`PRG Pattern`	POST-Process-Redirect-GET - prevents double submit	Always redirect after POST
`JSTL`	Tag library - replaces scriptlets in JSP	c:forEach not for scriptlet
`EL`	${user.name} = calls user.getName() automatically	${} = no Java in HTML
`Request scope`	request.setAttribute() - lives for 1 request (forward OK)	Single trip scope
`Session scope`	session.setAttribute() - lives until logout/timeout	User login data
`@WebFilter`	Intercepts requests - use for auth, logging, encoding	Bouncer at the door
`web.xml`	Deployment descriptor - servlet mapping, error pages, welcome files	App config file

Memory Tricks:

MVC = 'Man Very Carefully' separates concerns - Model (man), View (visuals), Controller (careful routing)

Controller is THIN - reads input, calls service, forwards to view. Nothing else!

View is DUMB - only displays. If you see SQL or business logic in JSP, it's wrong.

Service is the BRAIN - all business rules, validations, transactions live here

DAO is the HANDS - only it touches the database. No SQL anywhere else!

PRG = 'Please Redirect after Get' - always sendRedirect() after a POST submission

@RestController = @Controller + @ResponseBody - returns data, NOT a view name

The Golden Rule of MVC

```
Controller  ->  calls Service  ->  calls DAO  ->  talks to DB
Controller  ->  sets attributes  ->  forwards to View (JSP)
View (JSP)  ->  reads attributes  ->  renders HTML only

NEVER: SQL in Controller | NEVER: SQL in JSP | NEVER: HTML in Servlet
NEVER: Business logic in DAO | NEVER: HTTP code in Service/DAO
```